

Searching Using Regular Expressions

In this chapter, you'll learn how to use regular expressions within MySQL `WHERE` clauses for greater control over data filtering.

Understanding Regular Expressions

The filtering examples in the previous two chapters enabled you to locate data using matches, comparisons, and wildcard operators. For basic filtering (and even some not-so-basic filtering), this might be enough. But as the complexity of filtering conditions grows, so does the complexity of the `WHERE` clauses themselves—and this is where regular expressions become useful.

A regular expression is a special string (that is, a set of characters) that is used to match text. If you need to extract phone numbers from a text file, for example, you might use a regular expression. If you need to locate all files with digits in the middle of their names, you might use a regular expression. If you want to find all repeated words in a block of text, you might use a regular expression. And if you want to replace all URLs on a page with actual HTML links to those same URLs, you might use a regular expression (or two).

Regular expressions are supported in all sorts of programming languages, text editors, operating systems, and more. Savvy programmers and network managers have long regarded regular expressions as a vital component of their technical toolboxes.

Regular expressions are created using the regular expression language, a specialized language designed to do everything just discussed and much more. Like any other language, regular expression language has special syntax and instructions that you must learn.

Note

To Learn More Full coverage of regular expressions is beyond the scope of this chapter. While the basics are covered here, for a more thorough introduction to regular expressions, you might want to obtain a copy of my book *Learning Regular Expressions* (ISBN 9780134757063).

Using MySQL Regular Expressions

So what do regular expressions have to do with MySQL? As already explained, regular expressions are used to match text by comparing a pattern (the regular expression) with a string of text. MySQL provides rudimentary support for regular expressions with `WHERE` clauses, allowing you to specify regular expressions that are used to filter data retrieved by using `SELECT`.

Note

Just a Subset of the Regular Expression Language If you are already familiar with regular expressions, take note. MySQL supports only a small subset of what most regular expression implementations support, and this chapter covers most of what is supported.

This will all become much clearer with some examples.

Basic Character Matching

We'll start with a very simple example. The following statement retrieves all rows where the column `prod_name` contains the text 1000:

► Input

```
SELECT prod_name
FROM products
WHERE prod_name REGEXP '1000'
ORDER BY prod_name;
```

► Output

prod_name
JetPack 1000

► Analysis

This statement looks much like the ones that used `LIKE` in Chapter 8, “Using Wildcard Filtering,” except that the keyword `LIKE` has been replaced with `REGEXP`. This tells MySQL that what follows is to be treated as a regular expression (one that just matches the literal text 1000).

So, why bother using a regular expression? Well, in this example, regular expressions really add no value (and probably hurt performance), but consider this next example:

► Input

```
SELECT prod_name
FROM products
WHERE prod_name REGEXP '.000'
ORDER BY prod_name;
```

► Output

```

+-----+
| prod_name |
+-----+
| JetPack 1000 |
| JetPack 2000 |
+-----+

```

► Analysis

Here the regular expression `.000` is used. The period (`.`) is a special character in regular expression language. It means “match any single character,” and so both 1000 and 2000 match and are returned.

Of course, this particular example could also be accomplished by using `LIKE` and wildcards (refer to Chapter 8).

Note

LIKE Versus REGEXP There is one very important difference between `LIKE` and `REGEXP`. Look at these two statements:

```

SELECT prod_name
FROM products
WHERE prod_name LIKE '1000'
ORDER BY prod_name;

```

and:

```

SELECT prod_name
FROM products
WHERE prod_name REGEXP '1000'
ORDER BY prod_name;

```

If you were to try them both, you’d discover that the first one returns no data, and the second one returns one row. Why is this?

As you saw in Chapter 8, `LIKE` matches an entire column. If the text to be matched exists in the middle of a column value, `LIKE` will not find it and will not return the row (unless wildcard characters are used). `REGEXP`, on the other hand, looks for matches within column values, and so if the text to be matched exists in the middle of a column value, `REGEXP` finds it and returns the row. This is a very important distinction.

So can `REGEXP` be used to match entire column values (so that it functions like `LIKE`)? Actually, yes, if you also use the `^` and `$` anchors, as explained later in this chapter.

Tip

Matches Are Not Case-Sensitive Regular expression matching in MySQL is not case-sensitive; that is, either case will be matched. To force case-sensitivity, you can use the `BINARY` keyword, as in this example:

```
WHERE prod_name REGEXP BINARY 'JetPack .000'
```

Performing OR Matches

To search for one of two strings (either one or the other), you use the pipe character (`|`), as shown here:

► Input

```
SELECT prod_name
FROM products
WHERE prod_name REGEXP '1000|2000'
ORDER BY prod_name;
```

► Output

```
+-----+
| prod_name |
+-----+
| JetPack 1000 |
| JetPack 2000 |
+-----+
```

► Analysis

Here the regular expression `1000|2000` is used. `|` is the regular expression OR operator. It means “match one or the other,” and so both 1000 and 2000 match and are returned.

Using `|` is functionally similar to using OR statements in SELECT statements, with multiple OR conditions being consolidated into a single regular expression.

Tip

More Than Two OR Conditions You can specify more than two OR conditions. For example, you can use `'1000|2000|3000'` to match 1000 or 2000 or 3000.

Matching One of Several Characters

The character `.` matches any single character. But what if you want to match only specific characters? You can do this by specifying a set of characters enclosed within `[` and `]`, as shown here:

► Input

```
SELECT prod_name
FROM products
```

```
WHERE prod_name REGEXP '[123] Ton'
ORDER BY prod_name;
```

► Output

```
+-----+
| prod_name |
+-----+
| 1 ton anvil |
| 2 ton anvil |
+-----+
```

► Analysis

Here the regular expression `[123] Ton` is used. `[123]` defines a set of characters, and it says to match 1 or 2 or 3. In this case, both `1 ton` and `2 ton` match and are returned (and there is no `3 ton`).

As you have just seen, `[]` is another form of an `OR` statement. In fact, the regular expression `[123] Ton` is shorthand for `[1|2|3] Ton`, which would also work. But the `[]` characters are needed to define what the `OR` statement is looking for. To better understand this, look at this example:

► Input

```
SELECT prod_name
FROM products
WHERE prod_name REGEXP '1|2|3 Ton'
ORDER BY prod_name;
```

► Output

```
+-----+
| prod_name |
+-----+
| 1 ton anvil |
| 2 ton anvil |
| JetPack 1000 |
| JetPack 2000 |
| TNT (1 stick) |
+-----+
```

► Analysis

Well, this doesn't work. The two required rows are retrieved, but so are three others. This happens because MySQL assumes that you mean to match `1 ton` or `2 ton` or `3 ton`. The `|` character applies to the entire string unless it is enclosed with a set.

Sets of characters can also be negated. That is, you can tell MySQL to match anything *except* the specified characters. To negate a character set, place a `^` at the start of the set. So, whereas `[123]` matches characters 1, 2, or 3, `^[123]` matches anything except those characters.

Matching Ranges

You can use a set to define one or more characters to be matched. For example, the following set matches digits 0 through 9:

```
[0123456789]
```

To simplify this type of set, you can use - to define a range. The following is functionally identical to [0123456789]:

```
[0-9]
```

Ranges are not limited to complete sets, so [1-3] and [6-9] are valid ranges, too. In addition, ranges need not be numeric; for example, [a-z] will match any alphabetical character.

Here is an example:

► Input

```
SELECT prod_name
FROM products
WHERE prod_name REGEXP '[1-5] Ton'
ORDER BY prod_name;
```

► Output

```
+-----+
| prod_name |
+-----+
| .5 ton anvil |
| 1 ton anvil |
| 2 ton anvil |
+-----+
```

► Analysis

Here the regular expression [1-5] Ton is used. [1-5] defines a range, and this expression means “match 1 through 5.” In this case, three matches are returned. .5 ton is returned because 5 ton matches (without the . character).

Matching Special Characters

The regular expression language is made up of special characters that have specific meanings. You’ve already seen ., [], |, and -, and there are others, too. So if you need to match those characters, how can you do it? For example, if you want to find values that contain the . character, how can you search for it? Look at this example:

► Input

```
SELECT vend_name
FROM vendors
WHERE vend_name REGEXP '\.'
ORDER BY vend_name;
```

► Output

```
+-----+
| vend_name |
+-----+
| ACME      |
| Anvils R Us |
| Furball Inc. |
| Jet Set   |
| Jouets Et Ours |
| LT Supplies |
+-----+
```

► Analysis

This did not work. `.` matches any character, and so every row is retrieved.

To match special characters, you need to precede them with `\\`. So, for example, `\\-` means “find `-`,” and, as shown in this example, `\\.` means “find `.`”:

► Input

```
SELECT vend_name
FROM vendors
WHERE vend_name REGEXP '\\.'
ORDER BY vend_name;
```

► Output

```
+-----+
| vend_name |
+-----+
| Furball Inc. |
+-----+
```

► Analysis

This works. `\\.` matches `.`, and so only a single row is retrieved. This process is known as *escaping*, and all characters that have special significance within regular expressions must be escaped this way—including `.`, `|`, `[]`, and all of the other special characters used thus far.

`\\` can also be used to refer to metacharacters (that is, characters that have specific meanings), such as the ones listed in Table 9.1.

Table 9.1 White Space Metacharacters

Metacharacter	Description
<code>\\f</code>	Form feed
<code>\\n</code>	Line feed
<code>\\r</code>	Carriage return
<code>\\t</code>	Tab
<code>\\v</code>	Vertical tab

Tip

**To Match ** To match the backslash character itself (`\`), you need to use `\\`.

Note

`\ or \?` With most regular expression implementations, you use a single backslash to escape special characters in order to use them as literals. MySQL, however, requires two backslashes. (MySQL interprets one of the backslashes, and the regular expression library interprets the other one.)

Matching Character Classes

There are matches that you'll find yourself using frequently: digits, all alphabetical characters, all alphanumerical characters, and so on. To make working with these matches easier, you can use predefined character sets known as *character classes*. Table 9.2 describes the available character classes.

Table 9.2 Character Classes

Class	Description
<code>[:alnum:]</code>	Any letter or digit (same as <code>[a-zA-Z0-9]</code>)
<code>[:alpha:]</code>	Any letter (same as <code>[a-zA-Z]</code>)
<code>[:blank:]</code>	Space or tab (same as <code>[\t]</code>)
<code>[:cntrl:]</code>	ASCII control characters (ASCII 0 through 31 and 127)
<code>[:digit:]</code>	Any digit (same as <code>[0-9]</code>)
<code>[:graph:]</code>	Any printable character (same as <code>[:print:]</code> but excludes space)
<code>[:lower:]</code>	Any lowercase letter (same as <code>[a-z]</code>)
<code>[:print:]</code>	Any printable character
<code>[:punct:]</code>	Any character that is in neither <code>[:alnum:]</code> nor <code>[:cntrl:]</code>
<code>[:space:]</code>	Any white space character, including the space (same as <code>[\f\n\r\t\v]</code>)
<code>[:upper:]</code>	Any uppercase letter (same as <code>[A-Z]</code>)
<code>[:xdigit:]</code>	Any hexadecimal digit (same as <code>[a-fA-F0-9]</code>)

Matching Multiple Instances

All of the regular expressions used thus far attempt to match a single occurrence. If there is a match, the row is retrieved; if there is not a match, nothing is retrieved. But sometimes you'll require greater control over the number of matches. For example, you might want to locate all numbers, regardless of how many digits a number contains, or

you might want to locate a word but also want to be able to accommodate a trailing *s* if one exists, and so on.

These sorts of matches can be accomplished by using the regular expressions repetition metacharacters, listed in Table 9.3.

Table 9.3 Repetition Metacharacters

Metacharacter	Description
*	Zero or more matches
+	One or more matches (equivalent to {1, })
?	Zero or one match (equivalent to {0, 1})
{n}	Specified number of matches
{n, }	No fewer than a specified number of matches
{n, m}	Range of matches (m not to exceed 255)

Consider this example:

► Input

```
SELECT prod_name
FROM products
WHERE prod_name REGEXP '\\([0-9] sticks?\\)'
ORDER BY prod_name;
```

► Output

```
+-----+
| prod_name |
+-----+
| TNT (1 stick) |
| TNT (5 sticks) |
+-----+
```

► Analysis

The regular expression `\\([0-9] sticks?\\)` requires some explanation. `\\(` matches `(`, `[0-9]` matches any digit (1 and 5 in this example), `sticks?` matches `stick` and `sticks` (the `?` after the `s` makes that `s` optional because `?` matches 0 or 1 occurrence of whatever it follows), and `\\)` matches the closing `)`. Without the `?` metacharacter, it would be very difficult to match both `stick` and `sticks`.

Here's an example that tries to match four consecutive digits:

► Input

```
SELECT prod_name
FROM products
WHERE prod_name REGEXP '[:digit:]{4}'
ORDER BY prod_name;
```

► Output

```
+-----+
| prod_name |
+-----+
| JetPack 1000 |
| JetPack 2000 |
+-----+
```

► Analysis

As explained previously, `[[:digit:]]` matches any digit, and so `[[:digit:]]` is a set of digits. `{4}` requires exactly four occurrences of whatever it follows (in this case, any digit), and so `[[:digit:]]{4}` matches any four consecutive digits.

It is worth noting that when using regular expressions, there is almost always more than one way to write a specific expression. The previous example could also be written as follows:

► Input

```
SELECT prod_name
FROM products
WHERE prod_name REGEXP '[0-9][0-9][0-9][0-9]'
ORDER BY prod_name;
```

Anchors

All of the examples thus far have matched text anywhere within a string. To match text at specific locations, you need to use anchor metacharacters, as listed in Table 9.4.

Table 9.4 Anchor Metacharacters

Metacharacter	Description
<code>^</code>	Start of text
<code>\$</code>	End of text
<code>[[:<:]]</code>	Start of word
<code>[[:>:]]</code>	End of word

For example, what if you wanted to find all products that start with a number (including numbers that start with a decimal point)? A simple search for `[0-9\.\.]` (or `[[:digit:]]\.\.`) would not work because it would find matches anywhere within the text. The solution is to use the `^` anchor, as shown here:

► Input

```
SELECT prod_name
FROM products
WHERE prod_name REGEXP '^[0-9\.\.]'
ORDER BY prod_name;
```

► Output

```
+-----+
| prod_name |
+-----+
| .5 ton anvil |
| 1 ton anvil |
| 2 ton anvil |
+-----+
```

► Analysis

`^` matches the start of a string. Therefore, `^[0-9\\.]` matches `.` or any digit only if they are the first characters within a string. Without the `^`, four other rows would be retrieved, too (those that have digits in the middle).

Note

The Dual-Purpose `^` `^` has two uses. It can be used within a set (defined using `[` and `]`) to negate that set. Otherwise, it is used to refer to the start of a string.

Note

Making REGEXP Behave Like LIKE Earlier in this chapter, I mentioned that `LIKE` and `REGEXP` behave differently in that `LIKE` matches an entire string, and `REGEXP` matches substrings, too. When you use anchors, you can make `REGEXP` behave just like `LIKE` by simply starting each expression with `^` and ending it with `$`.

Tip

Simple Regular Expression Testing You can use `SELECT` to test regular expressions without using database tables. `REGEXP` checks always return 0 (for no match) or 1 (for a match). You can use `REGEXP` with literal strings to test expressions and to experiment with them. Here's an example:

```
SELECT 'hello' REGEXP '[0-9]';
```

This example would return 0 because there are no digits in the text `hello`.

Summary

In this chapter, you learned the basics of regular expressions and how to use them in MySQL `SELECT` statements via the `REGEXP` keyword.

Challenges

1. Use regular expressions to return every product whose name ends with a number.
2. In this chapter, you learned how to use `REGEXP` to match text containing digits. Can you figure out how to match only products with no digits in their names? Here's a hint: You can negate an entire match (as you learned in Lesson 7, "Advanced Data Filtering").
3. This final one is a little trickier. Some of the products listed in `products` have names that are made up of more than one word. Use regular expressions to return only the products with names made up of three or more words. Here's a hint: Look for spaces between the words.